

JomCode Coding Bootcamp

Pydantic and FastAPI

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Validate incoming workshop data using Pydantic.
- 02 Run a minimal FastAPI route and inspect /docs.
- 03 Separate request and response models so private fields stay private.

Plain meaning → technical view → use

PLAIN MEANING

Check outside data before application code trusts it.

TECHNICAL VIEW

Pydantic models validate request fields; FastAPI binds routes and response models.

WHEN YOU USE IT

Reject a capacity of zero or a timezone-free workshop date.

TIP

Open `/docs` on localhost; compare invalid and valid request bodies.

Install the API packages

TERMINAL >_ PowerShell / zsh

```
> python -m pip install fastapi uvicorn pydantic
```

Run from your project virtual environment.

Worked example 1: Strict capacity



chapter_17_pydantic_and_fastapi.py

```
1 from pydantic import BaseModel, Field
2 class Count(BaseModel):
3     capacity: int = Field(ge=1, le=500, strict=True)
4 print(Count(capacity=20).capacity)
```

PREDICT Will True count as one seat?

Example 1: output and reason

TERMINAL	>_	EXPECTED OUTPUT

20

WHY THIS HAPPENS

Strict validation rejects bool and string capacity values.

Exercise 1: Capacity boundary

BUILD

Accept 1 and 500. Reject 0, 501, True and "20".

CHECK

All six outcomes are explicit.

IF STUCK

Test both accepted and rejected input.

Worked example 2: Minimal route



chapter_17_pydantic_and_fastapi.py

```
1 from fastapi import FastAPI
2 from fastapi.testclient import TestClient
3 mini = FastAPI()
4 @mini.get("/health")
5 def health(): return {"status": "ok"}
6 print(TestClient(mini).get("/health").json())
```

PREDICT Does this route need a database?

Example 2: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
{'status': 'ok'}
```

WHY THIS HAPPENS

No. Learn request routing before adding persistence.

Exercise 2: PATCH semantics

BUILD

Accept one changed venue, but reject empty and explicit-null updates.

CHECK

venue accepted; {} and {venue:null} fail.

IF STUCK

Omitted and null fields mean different things.

Worked example 3: Public schema



chapter_17_pydantic_and_fastapi.py

```
1 from app.schemas import UserOut
2 print(sorted(UserOut.model_fields))
```

PREDICT Where is the password hash field?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
['email', 'full_name', 'id', 'role']
```

WHY THIS HAPPENS

Supabase Auth manages password data; the API profile response has no password field.

Exercise 3: Response filtering

BUILD

Create a tiny `response_model` route that excludes an internal field.

CHECK

JSON contains title and capacity only.

IF STUCK

The public model filters what the browser sees.

Checkpoint

PROVE IT

A learner predicts 422 for invalid capacity and can explain why UserOut has no password field.